使用 ElevenLabs 通过代码生成 Audiobook

结论

- `ElevenLabs API` 可以逐段生成音频
- 你把长文本切块后，可以**批量合成章节**
- 对长篇最关键的是用 **Request Stitching**
- 如果是纯 audiobook，我更建议用 `eleven_multilingual_v2`
- 如果要更戏剧化对白，可考虑 `eleven_v3`，但它**不支持** Request Stitching

推荐方案

1. 文本按 `章 -> 段` 切分
2. 每段调用 ElevenLabs TTS
3. 用 `previous_request_ids` 保持前后语气连续
4. 每章合并成一个 `mp3`
5. 最后输出整本书，或保留分章文件

为什么 ElevenLabs 适合 Audiobook

- 官方把 TTS 明确列为可用于 `audiobooks`
- `eleven_multilingual_v2` 官方写的是更适合长文本、稳定性更好
- 官方提供 `Request Stitching`，就是专门解决长文切块后语气断裂的问题

要注意的点

- `eleven_v3` 不支持 `Request Stitching`
- `request-id` 官方说明建议在 **2 小时内** 用于 stitching
- 免费层通过 API 访问 voice library 有限制
- 计费是按字符

最实用的代码路线

下面这版就是“文本切块 -> 生成 -> 拼章节”的最小可用脚本。

```
import os
import re
import subprocess
from pathlib import Path
from elevenlabs.client import ElevenLabs

API_KEY = os.getenv("ELEVENLABS_API_KEY")
VOICE_ID = "JBFqnCBsd6RMkjVDRZzb"  # 换成你的 voice_id
MODEL_ID = "eleven_multilingual_v2"
OUT_DIR = Path("output_audiobook")
OUT_DIR.mkdir(exist_ok=True)

client = ElevenLabs(api_key=API_KEY)

def split_text(text: str, max_chars: int = 3000):
    text = re.sub(r"\n{2,}", "\n\n", text.strip())
    paras = [p.strip() for p in text.split("\n\n") if p.strip()]
    chunks = []
    buf = ""

    for p in paras:
        if len(buf) + len(p) + 2 <= max_chars:
            buf = f"{buf}\n\n{p}".strip()
        else:
            if buf:
                chunks.append(buf)
            if len(p) <= max_chars:
                buf = p
            else:
```

```
                # 超长段再按句号拆
                sentences = re.split(r'(?<=[。！？.!?])', p)
                temp = ""
                for s in sentences:
                    if len(temp) + len(s) <= max_chars:
                        temp += s
                    else:
                        if temp:
                            chunks.append(temp.strip())
                        temp = s
                buf = temp.strip()

    if buf:
        chunks.append(buf)

    return chunks

def synthesize_chapter(chapter_title: str, text: str):
    chapter_dir = OUT_DIR / chapter_title
    chapter_dir.mkdir(parents=True, exist_ok=True)

    chunks = split_text(text)
    request_ids = []
    part_files = []

    for i, chunk in enumerate(chunks, start=1):
        out_file = chapter_dir / f"{i:03}.mp3"

        with client.text_to_speech.with_raw_response.convert(
            voice_id=VOICE_ID,
            model_id=MODEL_ID,
            text=chunk,
            output_format="mp3_44100_128",
            previous_request_ids=request_ids[-3:],  # 保留最近几个上下文
        ) as response:
            req_id = response._response.headers.get("request-id")
            if req_id:
                request_ids.append(req_id)

            audio = b"".join(response.data)
            out_file.write_bytes(audio)

        part_files.append(out_file)
```

```
        print(f"done: {out_file}")

    concat_txt = chapter_dir / "concat.txt"
    concat_txt.write_text(
        "\n".join([f"file '{f.resolve()}'" for f in part_files]),
        encoding="utf-8"
    )

    chapter_mp3 = OUT_DIR / f"{chapter_title}.mp3"
    subprocess.run([
        "ffmpeg", "-y",
        "-f", "concat", "-safe", "0",
        "-i", str(concat_txt),
        "-c", "copy",
        str(chapter_mp3)
    ], check=True)

    print(f"chapter built: {chapter_mp3}")
    return chapter_mp3


if __name__ == "__main__":
    sample_text = '''
    第一章

    这是第一段。

    这是第二段，内容稍长一些。你可以把整章文本放在这里，脚本会自动切块并生成音频。
    '''

    synthesize_chapter("chapter_01", sample_text)
## 依赖安装

`pip install elevenlabs`

如果你要合整本书，再把每章 mp3 继续 concat 一次就行。

----------

## 更稳的工程化建议

- 每章单独一个目录
- 每段保留原文本和音频文件
```

- 记录 request-id
- 失败重试只重跑失败段
- 人名地名先做 pronunciation 词典
- 先输出分章 mp3，最后再封装成 m4b

如果你不想全写代码

ElevenLabs 还有 Studio，官方支持：

- 导入整本书 / 文档
- 自动按章节处理
- 导出单文件或按章节 ZIP

但那是偏网页工作流，不是纯 API。

建议

- 自动化生产：API + 脚本
- 人工精修和出版前检查：Studio